

Session 5 Lab Report

Objective:

To develop a console-based Contact Manager in Kotlin using data classes, null safety handling, and higher-order functions for filtering contacts. The task improves logical structuring and reusable function design.

Kotlin Code:

```
data class Contact(
    val name: String,
    val phone: String?,
    val email: String?,
    val isFavorite: Boolean
)

fun addContact(list: MutableList<Contact>, contact: Contact) {
    list.add(contact)
}

fun filterContacts(
    contacts: List<Contact>,
    predicate: (Contact) -> Boolean
): List<Contact> {
    return contacts.filter(predicate)
}

fun displayContacts(contacts: List<Contact>) {
    if (contacts.isEmpty()) {
        println("No contacts found.")
        return
    }

    contacts.forEach {
        println("Name: ${it.name}")
        println("Phone: ${it.phone ?: "Not Available"}")
        println("Email: ${it.email ?: "Not Available"}")
        println("Favorite: ${if (it.isFavorite) "Yes" else "No"}")
        println("----- ")
    }
}

fun main() {
    val contactList = mutableListOf<Contact>()

    addContact(contactList, Contact("Chinmay"9876543210", "chinmay@email.com", true))
    addContact(contactList, Contact("Anu", null, "Anu@email.com", false))
    addContact(contactList, Contact("Aksha", "9123456780", null, true))

    println("All Contacts:")
    displayContacts(contactList)

    println("Favorite Contacts:")
    val favorites = filterContacts(contactList) { it.isFavorite }
    displayContacts(favorites)

    println("Contacts with Email:")
    val emailContacts = filterContacts(contactList) { it.email != null }
    displayContacts(emailContacts)
}
```

Minimal Explanation:

1. Contact data class stores contact details with nullable phone and email. 2. `addContact()` adds new contacts to the list. 3. `filterContacts()` uses a higher-order function to filter based on conditions. 4. `displayContacts()` safely handles null values using the Elvis operator (`?:`). 5. `main()` demonstrates filtering favorite and email contacts.